

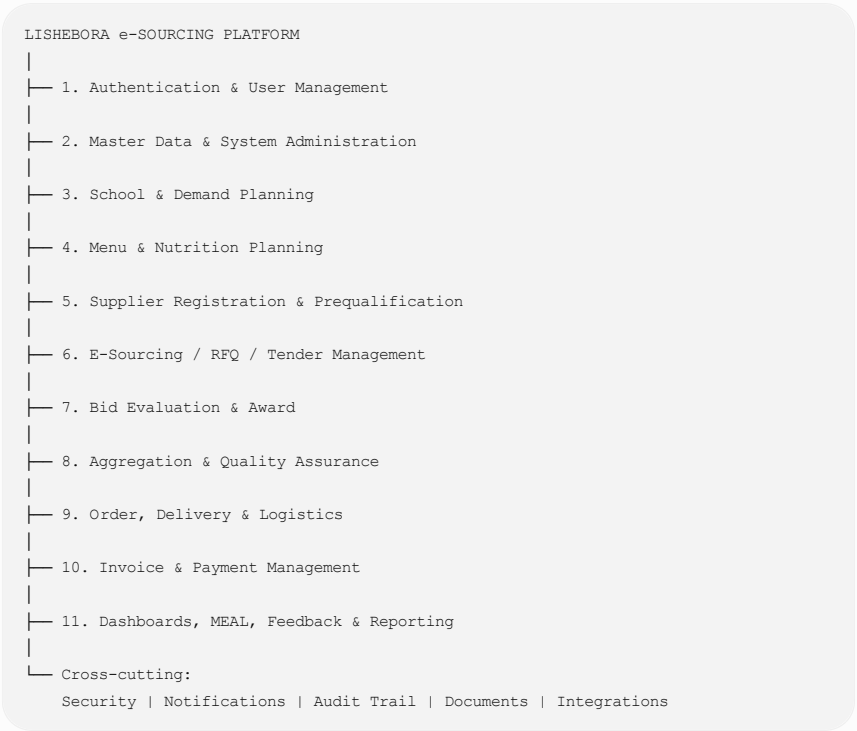
Yes. Based on the **LisheBora e-Sourcing Platform System Development Brief**, I would recommend defining the **system structure, modules, user roles, permissions, workflows, and access rules before development starts**. The brief already provides the foundation, including the end-to-end flow from school demand to supplier payment.

System development Brief

The important thing is to turn the brief into a **clear functional blueprint** that a developer can use to design the database, APIs, screens and permissions.

1. Recommended overall system structure

I would structure LisheBora into **11 major modules**:



This follows the brief's proposed modular, mobile-first and API-enabled architecture.

System development Brief

2. User roles and access

I recommend **not giving people access simply because they belong to an organisation**. Access should be based on **role + organisation + geography + responsibility**.

For example:

A County Procurement Officer can access procurement activities in their county, but should not automatically see another county's procurement.

A proposed role structure would be:

Role	Main responsibility
Super Administrator	Full system configuration
County Administrator	County-level administration and oversight
School Administrator / Headteacher	School information and demand
School Meals Committee	Menu, demand and delivery verification
Nutrition Officer	Menus and nutrition standards
Procurement Officer	Sourcing and procurement

Role	Main responsibility
Evaluation Committee Member	Bid evaluation
Approving Officer	Procurement approval
Supplier	Bidding, orders, invoices
Farmer Group / Cooperative	Aggregation and farmer records
Aggregator / Hub Manager	Aggregation, stock and dispatch
Quality Inspector	Inspection and quality clearance
Logistics Officer	Delivery and transport
Finance Officer	Invoice and payment processing
County Finance/Treasury	Payment approval/status
MEAL Officer	Monitoring and reporting
Project Manager	Programme-level oversight
Auditor	Read-only audit access
Helpdesk/Support	User support
Public/Guest	Limited public information

The original brief identifies the major user groups, including schools, county authorities, farmer groups, suppliers, quality officers, finance, MEAL and system administrators.

System development Brief

3. Detailed role access

A. Super Administrator

Access

Everything, but with strong audit controls.

Features

- Create/edit/deactivate users
- Assign roles
- Assign county/sub-county/school access
- Configure approval workflows
- Configure procurement thresholds
- Configure commodities
- Configure units of measurement
- Configure nutrition parameters
- Configure quality standards
- Configure supplier categories
- Configure notification rules
- Configure system settings
- Manage integrations
- View system-wide audit logs
- Manage reporting periods
- Configure dashboards
- Manage document templates
- Manage system backup settings

Important restriction

The Super Admin should **not be able to silently alter transactional records**.

If something is changed, the audit trail should show:

Who → What → Previous value → New value → Date/time → Reason

This is particularly important because the brief requires complete audit trails and controlled access.

System development Brief

4. County Administrator

Dashboard

County Dashboard

Schools	126
Active Suppliers	84
Open RFQs	17
Orders	93
Deliveries	81
Pending Approvals	12
Outstanding Invoices	24

Access

Can:

- View county schools
- View county demand
- Review procurement plans
- Approve designated plans
- View suppliers operating in county
- Manage supplier eligibility where authorized
- Monitor procurement
- View county budgets
- Monitor deliveries
- Monitor payment status
- View complaints
- View county performance
- Generate reports

Cannot:

- Modify another county
- Evaluate bids where they are the approving authority
- Approve their own procurement action
- Access confidential bids before the closing date unless explicitly authorized

5. School Administrator / Headteacher

This should be one of the **simplest interfaces in the system**.

Dashboard

My School

Learners	842
Feeding Days	20
Current Stock	8 items
Pending Orders	4
Expected Deliveries	3
Deliveries Today	1
Outstanding Issues	2

Features

- Manage school profile
- Enter enrolment

- Enter expected attendance
- Enter feeding days
- View approved menus
- Enter stock balances
- Generate food requirements
- Submit demand
- View procurement status
- View awarded supplier
- View purchase orders
- Confirm deliveries
- Accept/reject quantities
- Record shortages
- Record damaged goods
- Upload delivery evidence
- Submit supplier feedback
- Submit complaints

The brief specifically requires schools to enter enrolment, meal plans, stock and requirements and confirm deliveries.

System development Brief

6. School Meals Committee

This can either be a separate role or a sub-role under the school.

Features

- Review proposed menu
- Review school demand
- Confirm food requirements
- Participate in delivery verification
- Record food quality concerns
- Provide supplier feedback
- Submit complaints
- View school procurement history

Important: they should not have access to financial approval or supplier bid evaluation unless specifically assigned another role.

7. Nutrition Officer

Features

- Create nutrition standards
- Configure approved food groups
- Configure portion sizes
- Review menus
- Approve menus
- Identify nutritional gaps
- Monitor dietary diversity
- View nutrition dashboards
- Monitor commodity substitutions
- Generate nutrition reports

The system should calculate commodity requirements based on menus, feeding days, enrolment and standard portions.

System development Brief

8. Procurement Officer

This is a **core role**.

Features

- View approved demand
- Create sourcing events
- Create RFQs
- Create tender lots
- Define quantities
- Define specifications
- Define delivery locations
- Define deadlines
- Define eligibility criteria
- Invite eligible suppliers
- Send notifications
- Respond to supplier clarifications
- Issue amendments
- Close procurement
- Generate purchase orders
- Monitor procurement cycle

Cannot

The Procurement Officer should **not approve their own procurement**.

This separation is explicitly required in the brief: initiation, evaluation, approval, receipt and payment should be separated.

System development Brief

9. Supplier

The supplier portal should be completely different from the internal government/school portal.

Supplier Dashboard

My Business

Profile:	90% Complete
Eligibility:	Approved
Open RFQs:	6
My Bids:	4
Awards:	2
Orders:	3
Deliveries:	2
Invoices:	2
Payments:	1

Features

Registration

- Business profile
- Supplier type
- Location
- Commodities
- Production capacity
- Documents
- Tax information
- Certifications
- Bank/mobile money details
- Storage capacity
- Transport capacity
- Ownership information
- Gender/age/disability information
- Farmer group membership
- Previous experience

Procurement

- View opportunities

- Download specifications
- Ask questions
- Submit quotation
- Submit bid
- Upload documents
- Track bid status
- Receive award notification
- Accept purchase order
- View order
- Update fulfilment status

Finance

- Create invoice
- Submit invoice
- Track invoice
- Track payment
- View payment history

The brief already identifies these supplier registration and bidding requirements.

System development Brief

10. Farmer Group / Cooperative

This is important if the platform is supposed to strengthen local producer participation.

Features

- Register cooperative/group
- Register members
- Record farmer commodities
- Record production capacity
- Record expected harvest
- Record available quantities
- Record aggregation
- Generate batch/lot numbers
- Record farmer contribution
- Track deliveries
- View orders
- View payments
- Track farmer participation

Farmer-level traceability

For example:

```
School
↓
Purchase Order
↓
Supplier
↓
Aggregation Hub
↓
Batch #RB-2026-00452
↓
Cooperative
↓
Farmer A - 250 kg
Farmer B - 180 kg
Farmer C - 300 kg
```

That would support the traceability requirement in the brief.

System development Brief

11. Aggregator / Hub Manager

Features

- View incoming orders
- Record produce received
- Record farmer/source
- Record commodity
- Record variety
- Record quantity
- Create batch
- Record grading
- Record rejected quantity
- Record accepted quantity
- Manage inventory
- Allocate produce to school orders
- Prepare dispatch
- Print dispatch documents
- Track stock

12. Quality Inspector

This needs to be a **separate role**.

Mobile interface

Inspection

Order: LB-PO-000245
Commodity: Rice
Supplier: XYZ Cooperative
Quantity: 5,000 kg

- ✓ Quantity
- ✓ Packaging
- ✓ Moisture
- ✓ Cleanliness
- ✓ Grade
- ✓ Food Safety

[ACCEPT]
[REJECT]
[ACCEPT PARTIALLY]

Features

- View inspection assignments
- Inspect deliveries
- Record test results
- Upload photographs
- Record accepted quantity
- Record rejected quantity
- Record reason for rejection
- Issue corrective action
- Approve quality clearance
- View inspection history

The brief specifically proposes mobile inspection, photographs and digital clearance.

System development Brief

13. Logistics Officer

Features

- View approved orders
- Consolidate deliveries
- Plan routes

- Assign vehicle
- Assign driver
- Create dispatch note
- Schedule delivery
- Track delivery status
- Record delays
- Record shortages
- Record damaged goods
- Record substitutions
- Capture proof of delivery

GPS can be added later; the brief identifies digital dispatch and delivery confirmation as the minimum pilot requirement.

System development Brief

14. Finance Officer

Dashboard

Finance

Invoices awaiting verification	18
Invoices awaiting approval	9
Payments processing	15
Overdue payments	7
Total outstanding	KSh 4.8M

Features

- View accepted deliveries
- View purchase order
- Verify invoice
- Three-way matching
- Record deductions
- Approve invoice
- Submit payment
- Record payment reference
- Track payment
- Reconcile payments
- Handle partial payments
- View outstanding obligations
- Generate finance reports

The three-way match should be:

```
Purchase Order
+
Goods Received
+
Invoice
↓
Three-Way Match
↓
Payment Approval
```

This directly follows the brief.

System development Brief

15. Evaluation Committee Member

This role requires **very strong confidentiality controls**.

Features

- View assigned procurement
- Open bids only after deadline
- Review technical documents

- Score bids
- Enter comments
- Declare conflict of interest
- Submit evaluation
- View own evaluation

Should NOT see

- Other evaluator's scores before submission
- Confidential bids before closing
- Payment information
- Unrelated procurements

After all evaluators submit:

```

Evaluator A → 78
Evaluator B → 82
Evaluator C → 76
      ↓
    Consolidated Score
      ↓
    Approval Process
  
```

The system should calculate consolidated scores while retaining the complete audit trail.

System development Brief

16. Approving Officer

Features

- View procurement recommendation
- Review evaluation
- Review budget
- Review supplier
- Approve
- Reject
- Return for correction
- Provide comments
- Digitally sign/approve
- View approval history

Critical rule

The approval workflow should be configurable.

For example:

```

Procurement Officer
      ↓
Evaluation Committee
      ↓
Procurement Manager
      ↓
County Approval
      ↓
Finance/Budget Approval
      ↓
Purchase Order
  
```

But the exact hierarchy should be configurable because the brief says thresholds and approval levels must be confirmed during inception.

System development Brief

17. MEAL Officer

This role should have a **read-heavy analytical dashboard**.

Features

- View all programme indicators
- Track procurement cycle time
- Track local sourcing
- Track women/youth/PWD participation
- Track farmers reached
- Track delivery performance
- Track rejected quantities
- Track payment turnaround
- Track grievances
- Track school participation
- Export Excel
- Export PDF
- Generate donor reports

The brief provides a strong set of MEAL indicators that should become system-generated metrics rather than manually calculated reports.

System development Brief

18. Project Manager

Features

The Project Manager should have a **programme-wide view**:

- All counties
- All schools
- Suppliers
- Procurement
- Delivery
- Payments
- Quality
- Inclusion
- Complaints
- KPIs
- Risks
- Budget utilisation
- Performance trends

But they should generally **not edit transactional data**.

Their role is primarily:

Monitor → Analyse → Intervene → Report

19. Auditor

I strongly recommend adding this role even though it is not explicitly listed as a primary user group.

Access

Read-only.

Can access:

- User activities
- Procurement history
- Bid history
- Evaluation records
- Approval history
- Purchase orders
- Delivery records
- Quality records
- Invoices

- Payments
- Audit logs
- Changes to master data

Cannot change anything.

20. Helpdesk / Support Officer

Features

- View user account
- Reset password
- Assist registration
- View error logs
- Create support tickets
- Resolve user issues
- Escalate technical issues

They should **not** be able to view sensitive financial information unnecessarily.

21. The most important part: permission architecture

I would not build permissions as simply:

Admin / User

Instead use:

Role + Scope + Action

For example:

PROCUREMENT OFFICER
+
COUNTY = Nakuru
+
ACTION = Create RFQ

Another:

SCHOOL ADMIN
+
SCHOOL = ABC Primary
+
ACTION = Confirm Delivery

This gives much better security.

22. Recommended permission matrix

Module	School	Supplier	Aggregator	Procurement	Quality	Finance	County
School Profile	E	-	-	V	-	-	V
Demand Planning	E	-	-	V	-	V	A
Menu	E	-	-	V	-	-	A
Supplier Profile	-	E	E	V/A	V	-	A
RFQ	V	V	V	E	-	V	A

Module	School	Supplier	Aggregator	Procurement	Quality	Finance	County
Bids	-	E	E	V	-	-	V
Evaluation	-	-	-	E	V	-	A
Award	-	V	V	E	-	V	A
Aggregation	-	E	E	V	E	-	V
Quality	V	V	E	V	E	-	V
Delivery	E	E	E	V	E	V	V
Invoice	-	E	E	V	-	E/A	V
Payment	-	V	V	-	-	E/A	V
Reports	V	V	V	V	V	V	V
Audit Log	-	-	-	V	V	V	V

E = Edit/Create

V = View

A = Approve/Administer

The exact permissions should be validated during inception, but this is a good development starting point.

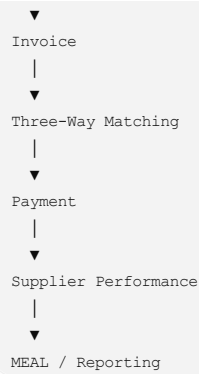
23. Core workflow

I would make the entire system revolve around one transaction lifecycle:

```

SCHOOL
|
▼
Demand Planning
|
▼
Demand Approval
|
▼
Procurement / RFQ
|
▼
Supplier Bidding
|
▼
Bid Evaluation
|
▼
Approval
|
▼
Purchase Order
|
▼
Aggregation
|
▼
Quality Inspection
|
▼
Dispatch
|
▼
Delivery
|
▼
School Acceptance
|

```



This reflects the end-to-end workflow already specified in the brief.

System development Brief

24. Status engine

One thing I would add before development is a **standard status engine**.

For example:

Procurement

```
DRAFT
↓
PENDING APPROVAL
↓
APPROVED
↓
PUBLISHED
↓
OPEN
↓
CLOSED
↓
UNDER EVALUATION
↓
RECOMMENDED
↓
APPROVED FOR AWARD
↓
AWARDED
↓
PO ISSUED
↓
COMPLETED
```

Order

```
DRAFT
→ APPROVED
→ CONFIRMED
→ PROCESSING
→ READY FOR DISPATCH
→ DISPATCHED
→ IN TRANSIT
→ DELIVERED
→ ACCEPTED
→ PARTIALLY ACCEPTED
→ REJECTED
→ CLOSED
```

Invoice

```
DRAFT
→ SUBMITTED
→ UNDER VERIFICATION
→ MATCHED
→ APPROVED
→ PAYMENT PROCESSING
→ PAID
```

→ PARTIALLY PAID
→ REJECTED

This prevents developers from creating inconsistent statuses across modules.

25. Notifications

The system should have a central notification engine.

Notifications through

- In-app
- SMS
- Email

Examples

Supplier

New RFQ available.

School

Delivery scheduled for tomorrow.

Procurement Officer

RFQ closes in 24 hours.

Approving Officer

Procurement approval required.

Finance

Invoice awaiting verification.

Supplier

Invoice approved.

Supplier

Payment completed.

The brief explicitly proposes SMS/email notifications.

System development Brief

26. Documents module

I would make documents a **central service**, rather than allowing every module to manage documents differently.

```
DOCUMENT MANAGEMENT
|
├─ Supplier Documents
├─ Tax Documents
├─ Certifications
├─ RFQ Documents
├─ Bid Documents
├─ Evaluation Documents
├─ Purchase Orders
├─ Delivery Notes
├─ Inspection Reports
├─ Invoices
├─ Payment Documents
└─ Grievance Evidence
```

Every document should have:

- Owner
- Document type

- Upload date
- Expiry date
- Version
- Status
- Related transaction
- Uploaded by
- Audit history

This is particularly important because supplier documents require expiry dates and renewal alerts.

System development Brief

27. Database structure

Before coding, I would also define the core entities.

Something roughly like:

```
Users
Roles
Permissions
Organizations
Counties
SubCounties
Schools
SchoolClusters

Students/Enrolment
Menus
MenuItems
NutritionStandards
Commodities
CommoditySpecifications
Units

Suppliers
SupplierDocuments
SupplierCommodities
SupplierCapacities
FarmerGroups
Farmers
AggregationHubs

DemandPlans
DemandItems
ProcurementPlans

SourcingEvents
SourcingLots
Bids
BidItems
BidDocuments
Evaluations
EvaluationScores
ConflictDeclarations
Awards

PurchaseOrders
PurchaseOrderItems

Batches
BatchItems
QualityInspections
QualityResults

Deliveries
DeliveryItems
Dispatches
ProofOfDelivery

Invoices
InvoiceItems
Payments
PaymentTransactions
```

Complaints
Feedback
SupplierPerformance

Notifications
Documents
AuditLogs
SystemSettings

That should be designed properly before development begins.

28. Dashboard structure

I would not give everyone the same dashboard.

School

"What do I need to do today?"

Supplier

"What opportunities and payments do I have?"

Procurement Officer

"What procurement actions require attention?"

Finance

"What invoices/payments require action?"

County Manager

"What is happening across my county?"

Project Manager

"Is the programme performing?"

MEAL

"What does the data tell us?"

Administrator

"Is the system functioning and properly configured?"

That distinction will make the platform much easier to use.

29. MVP — what I would actually develop first

I would **not develop everything in the brief at once**.

The brief itself proposes an MVP covering user management, demand planning, supplier registration, RFQs, bidding, evaluation, awards, delivery confirmation, invoice tracking, notifications and basic dashboards.

System development Brief

I would therefore make Phase 1:

MVP

1. Authentication
2. Role-based access
3. User management
4. School management
5. Supplier registration
6. Commodity master
7. Demand planning
8. Procurement plan
9. RFQ
10. Supplier bidding

11. Bid evaluation
12. Approval workflow
13. Purchase order
14. Delivery confirmation
15. Invoice tracking
16. Payment status
17. Notifications
18. Audit trail
19. Basic dashboards
20. Basic reports

Phase 2

- Aggregation
- Advanced quality management
- Advanced logistics
- Nutrition analytics
- Supplier performance
- Grievance management
- Advanced MEAL

Phase 3

- Forecasting
- GPS
- Mobile app/PWA enhancements
- Government integrations
- Banking/mobile-money integration
- Advanced analytics
- AI-assisted demand forecasting

This keeps the initial system manageable while preserving the architecture for future expansion.

30. One major recommendation before development

I would turn your current document into a **System Requirements Specification (SRS)** before giving it to developers.

The current brief is a very good **conceptual/business requirements document**, but developers will still need answers to questions such as:

- Exactly what screens are required?
- What fields are on each screen?
- Who can create/edit/approve each record?
- What happens after rejection?
- What are the statuses?
- What triggers a notification?
- What data can each role see?
- What happens when a document expires?
- What happens when a supplier partially delivers?
- What happens when a bid is tied?
- Can an approval be withdrawn?
- How are amendments handled?
- How are partial payments handled?
- What happens when a school rejects a delivery?
- Which actions require two-person approval?
- Which records can never be deleted?
- What are the exact audit requirements?
- What APIs will be required?

The existing brief already identifies several of these business rules—for example, separation of procurement functions, budget controls, audit trails, delivery acceptance before invoicing, and mandatory conflict-of-interest declarations.

My recommended development documentation hierarchy

```
graph TD; 01[01. BUSINESS REQUIREMENTS] --> 02[02. SYSTEM REQUIREMENTS SPECIFICATION]; 02 --> 03[03. USER ROLES & PERMISSION MATRIX]; 03 --> 04[04. USER JOURNEY / PROCESS FLOWS]; 04 --> 05[05. FUNCTIONAL MODULE SPECIFICATIONS]; 05 --> 06[06. DATABASE / ERD DESIGN]; 06 --> 07[07. API SPECIFICATION]; 07 --> 08[08. UI/UX WIREFRAMES]; 08 --> 09[09. SECURITY & DATA PROTECTION DESIGN]; 09 --> 10[10. TESTING / ACCEPTANCE CRITERIA]; 10 --> 11[11. DEVELOPMENT]; 11 --> 12[12. PILOT];
```

I would strongly recommend doing these first before writing the actual code. Your current brief is already a solid starting point; the next step should be to convert it into a **developer-ready system blueprint**, especially the **role-permission matrix + module-by-module features + database entities + workflows + screen list + approval rules**.

Sources